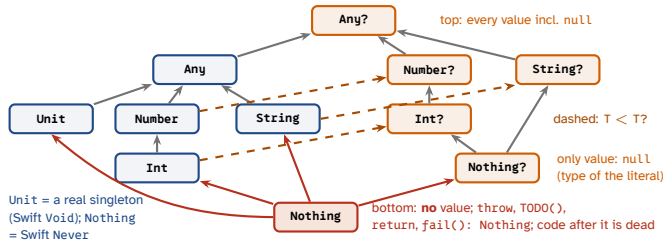
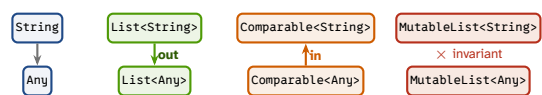


In one line: Every Kotlin type has a **nullable twin**; the hierarchy is a lattice with `Any?` on top and `Nothing` (no values) at the bottom, so `throw`, `TODO()` and `return` fit any expected type. Generics are **invariant** unless declared `out/in` at the class (declaration site) or projected at a use; on the JVM type arguments are **erased**, so runtime checks see only `List<*>` — unless an inline function makes `T` **reified**.

The lattice — arrow = “is a subtype of”



Variance — what the arrow does to C<_>



Kotlin	Java	you may	PECS role
<code>C<out T></code>	<code>C<? extends T></code>	read T, write nothing	producer
<code>C<in T></code>	<code>C<? super T></code>	write T, read Any?	consumer
<code>C<*></code>	<code>C<?></code>	read upper bound, write nothing	unknown
<code>C<T></code>	<code>C<T></code>	both — invariant	both

Kotlin emits wildcards for parameters only (not returns, not final types);
 @JvmSuppressWildcards/@JvmWildcard override it.

Top, bottom, platform

- `Nothing` makes failure branches type-check: `val n: String = x ?: fail("no").emptyList()` is an object `EmptyList : List<Nothing>` — one instance fits every `List<T>` because `List` is `out`. Same trick: `data object Loading : Load<Nothing>`.
- Default upper bound of `<T>` is `Any?`: `fun <T> f(t: T)` accepts `null`. `<T : Any>` forbids it. `T & Any = definitely non-nullable (1.7)` — e.g. to override Java's `@NotNull T`.
- Platform type** `T!` (from un-annotated Java) = “`T` or `T?`, you decide”. `@Nullable/@NonNull` (JSpecify, AndroidX, JetBrains) turn it into a real type. Declare the type explicitly at the boundary.

Smart casts — exactly when

- Stable** values only: `val` locals; `val` properties that are not open, have no custom getter and live in the **same module**; `var` locals not changed between check and use (and not captured by a lambda that changes them). Never: `var` properties, delegated properties.
- K2 (2.0)** adds: a check stored in a local (`val isCat = a is Cat`), `a is B || a is C` → closest common supertype, captured vars inside `inline` lambdas, info carried into `catch/finally`.
- Contracts** teach the compiler: `requireNotNull`, `check`, `isNullOrEmpty` smart-cast after the call; `run/let` promise calls `inPlace`. Your own contract `{}` still needs `@OptIn(ExperimentalContracts::class)`.

Example

```
fun fail(msg: String): Nothing = throw IllegalStateException(msg)
sealed interface Res<out T> { // out: Res<Nothing> fits
    data class Ok<T>(val v: T) : Res<T>
    data class Err(val e: Throwable) : Res<Nothing>
}
fun copy(from: Array<out Any>, to: Array<in Any>) { /*...*/ } // use-site
fun <T> max(xs: List<T>): T where T : Comparable<T>, T : Any = xs.max()
fun <T> T.orThrow(): T & Any = this ?: fail("null") // DNN type
inline fun <reified T> Any?.asOrNull(): T? = this as? T
val t = typeOf<List<String>>() // reified: keeps the type ARGUMENT
if (x is List<*>) { // is List<String> = compile error
    @JvmInline value class Email(val raw: String) {
        init { require('@' in raw) } // validated, zero-cost when unboxed
    }
}
fun interface Rule { fun ok(s: String): Boolean }
val notBlank = Rule { it.isNotBlank() } // SAM conversion
```

Erasure & reified

- JVM keeps only the class: `is List<String> = error`, as `List<String> = unchecked warning` — the CCE fires later, at the first element read.
- reified (inline only): the call site pastes the real class, so is `T, T::class, typeOf<T>()` work. `T::class.java` still drops `arguments` (`List<User> → List`); `typeOf` keeps them.
- Erased overloads clash (`f(List<String>)/f(List<Int>)`) → `@JvmName`.

Variance in practice

- `out T: T` only in `out` positions (returns, `val`); escape hatch `List.contains(e: @UnsafeVariance E)`. `Array<T>` is invariant (mutable), so `copy(from: Array<out Any>)` takes an `Array<String>` but cannot write into it.
- `Star: Foo<out T : U> → Foo<*>` reads `U`; `Foo<in T> → Foo<*> = Foo<in Nothing>`: writes nothing.

Sealed, value, alias, SAM

- sealed**: direct subclasses in the same **module + package**; when on `sealed/enum/Boolean` must be exhaustive (statements too, since 1.7) — skip `else` so a new subclass breaks the build.
- value class**: one `val`, unboxed; **boxed** as nullable, generic `T`, interface/`Any`; no `===`; functions taking one get a **mangled JVM name** (`send-<hash>`).
- typealias** = new name, same type, no safety. **fun interface**: one abstract method → lambda converts (Java SAMs always did).

Traps

- !** `fun <T> f(t: T): t` may be `null` — the bound is `Any?`.
- !** `val s = javaObj.name` infers `String!`: NPE at first use, far away.
- !** No smart cast on open `val`/other-module `val` — copy to a local.
- !** `List<Email>` of a value class = boxed elements; saving gone.

Remember: `Any?` top · `Nothing` bottom · `out` produces, `in` consumes · `erased` unless `reified`.